

6. REST 서비스 생성하기

1.1 REST 컨트롤러 작성하기

책) : Angular Framework 사용해서 SPA로 Front-end 구축.
새로운 브라우저 UI가 2장 페이지 대체.

↳ 타고 데이터 저장 및 가져오기 위해
Angular 기반의 UI와 통신하는 REST API 생성.

용어정리)

REST API란?

: REpresentational State Transfer 이 약자

: Web 기반의 자원에 대해 HTTP 프로토콜을 활용하여
데이터 요청하고 응답 받는 방식.

: 이 스타일 따르는 RESTful API 라고 한다.

REST API 개념.

1) 자원

: URI (Uniform Resource Identifier)로 식별하는 데이터. ex) /users, /post1

2) 표현

: 자원의 상태를 JSON, XML 등으로 표현해 전달.

3) 상태전이

: Client는 서버의 자원 상태를 HTTP 요청으로 바꾸고, 응답 통해
응답 통해 새로운 상태 얻음.

4) 무상태성

: 요청 간 서버가 클라이언트의 상태 저장 X. 모든 요청 독립적

REST API의 구성요소

1) HTTP Method

: GET, POST, PUT, PATCH, DELETE => 자원에 대한 행위 명시

2) URI

: 어떤 자원을 대상으로 하는지 식별

=> /users, /users/123

3) HTTP 상태코드

200: 성공 201: Created, 자원생성 완료
400: Bad Request, 잘못된 요청 401: Unauthorized, 인증 실패
404: Not found, 자원 X
500: Internal Server Error, 서버 오류

REST API 가 어떻게 쓰이나?

=> 보통 프론트/백 분리된 구조.

1) Front-end (vue.js, React ...)

: 사용자가 /user/123 이 들어가면 JS가 API 호출.

=> GET https://api.example.com/users/123

2) Back-end

: Server는 사용자 정보 DB에서 꺼내 JSON으로 응답.

```
=> {  
  "id": _____  
  "name": _____  
  "join_at": _____  
}
```

용어정리 2)

SPA vs MPA.

- SPA (Single Page Application)
- MPA (Multi Page Application)

1) SPA

- : 페이지 1개로 모든 화면 구성
- : 첫 페이지 로딩 후, JS가 동작으로 View만 교체
- : 페이지 이동 X, Router가 동작.

2) MPA

- : 여러 개의 HTML 페이지로 구성
- : 사용자 요청마다 전체 새로 로딩
- : 실제로 다른 URL로 이동하며 새로 고침.

	SPA	MPA
초기 로딩 속도	↓ (JS파일 큼)	↑
페이지 전환 속도	↑ (view만 교체)	↓ (전체 페이지 새로 요청)
서버 요청	API 요청만	페이지 + 리소스 제 요청.
개발 방식	Client 중심 (Vue, React...)	Server 중심. (JSP, Thymeleaf...)
SEO	불리 - JS 렌더링	유리
상태관리	Client에서 직접관리	페이지마다 초기화,
보안	JS가 많아 Client 공격면 넓음	요청마다 서버 처리 중심.

용어정리 3)

Front-end 기서 **Router**. (Vue Router, React Router...)

: 브라우저 주소(URL)에 따라 보여줄 페이지(view)를 결정하는 역할을 하는 JavaScript 라이브러리.

역할

- : URL이 바뀌면, 전체 페이지를 새로고침 없이 화면 구성만 바껴줌.
- : 사용자에게는 진짜 페이지가 이동하는 것 처럼 보이지만 실제로는 SPA - 한 페이지 안에서 View가 바뀐 것 뿐.

ex)

```
import Home from './Home.vue'
import About from './About.vue'
```

```
export default [
```

```
{ path: '/', component: Home },
```

```
{ path: '/about', component: About } : 사용자가 /about URL에 들어오면
Vue Router가 About.vue를
화면에 보여줌.
```

```
]
```

: 실제로는 HTML 새로 요청X.

Angular 클라이언트 코드

→ HTTP 요청 통해 REST API로 통신.

Spring MVC는 다양한 타입의 HTTP 요청에 사용되는 다른 액션 제공

L ① Get Mapping

: HTTP GET 요청, 리소스 읽기

L ② Post Mapping

: HTTP POST 요청, 리소스 생성

L ③ Put Mapping

: HTTP PUT 요청, 리소스 변경. ... 전체 자원 교체 (replace)

L ④ Patch Mapping

: HTTP PATCH 요청, 리소스 변경. -- 일부 자원 수정 (update)

L ⑤ Delete Mapping

: HTTP DELETE 요청, 리소스 삭제

L ⑥ Request Mapping

: 더욱적 요청처리. HTTP 메소드 method 특성에 지정.

⇒ 가장 최근에 생성된 태그 가져오는 간단한 REST 엔드 포인트 생성.

< Spring boot 3.x 버전 >

3

```
public class DesignTacoController {
```

```
private final TacoRepository tacoRepo;
```

```
public DesignController (TacoRepository tacoRepo) {
```

```
this.tacoRepo = tacoRepo;
```

3

Iterable <Taco> 끼어 변경. Spring boot 3.x 부터 Collection Model <Entity Model
<>> 3 작성해야함.

② Get Mapping ("/recent")

```
public CollectionModel < EntityModel < Taco > > recentTacos () {
```

Page Request page = Page Request. of (0, 2, Sort. by ("created At"))

descending(): 내림차순 최상순 정렬 descending()): 쿼리 만든 것중 12개만 보여줌. 0: 첫페이지 번호
 ↗ 페이지 단위로 데이터 조회 12: 한 페이지에 보여줄 갯수.

List< Taco > tacos = tacoRepo.findAll(page).getContent(); 타코 리스트 가져오기
→ 결과에서 실제 데이터 꺼내기.

List <Entity Model <Taco >> tacoResources = tacos.stream() 리스트 하나씩
꺼내기 위한 준비

라스트에 있는 각 데이터, `map(taco → Entity Model of (taco, Entity Model of (taco, link): taco 링크 포장`
처리해서 시킨 것

linkTo (methodOn (Design Taco Controller, class) linkTo(...) + methodOn(...)) : 링크 생성

```
recentTacos().withSelfRel().collect(Collectors.toList());
```

포장한 12개짜 타코
3개 1개로 포장.

return Collection Model. of $\square$ tacoResources,

```
linkTo (methodOn (DesignTacoController class). recentTacos()).withSelfRef()
```

} EntityModel.of(taco, linkTo(..), withSelfRel()) : 객체 자체 프로그래밍 함수.

,

링크만들기

2 링크가 자신 가리키도록 이름 붙임.

/design / recent면 recent 새서 왔다고 알린다.

CollectionModel < EntityModel < Taco > >

- EntityModel<T> : 객체 1개 + 링크 1개 포장

- Collection Model (Entry Model (T)) : 객체 리스트 + 링크 포장.

link To method On (DesignTaco Controller.class). recentTaco()

: 지금 이 클래스 안의 있는 recentTacos() 메소드 가르침

L: 이 메소드가 실행되는 웹 주소(URL) 만들어줌.

6.1.1 서버에서 데이터 가져오기

: Latest Designs 링크 클릭 시 최근에 생성한 타코 내역 보여주기.

↳ 최근에 생성된 타코 보여주는 RecentTacos Component를 앵글러 코드에 정의.

- 타코 디자인 API 요청 처리하는 REST 사용 컨트롤러.

```
package tacos.web.api;
```

```
:
```

→ 메소드가 리턴하는 값은 view에 이동x, HTTP 응답바디에 들어감.

@RestController : REST API용 컨트롤러임을 명시 (@Controller + @ResponseBody)

@RequestMapping (path = "/design", produces = "application/json")

@CrossOrigin (origins = "*")

```
public class DesignTacoController {
```

```
    private TacoRepository tacoRepo;
```

@Autowired 관계가 지켜짐.

EntityLinks entityLinks; : spring boot 3.4.3은 작전 X.

```
public DesignTacoController (TacoRepository tacoRepo) {
```

```
    this.tacoRepo = tacoRepo;
```

```
}
```

@GetMapping ("/recent")

→ CollectionModel <EntityModel <Taco>>로 바뀌어야 함.
HATEOAS 응답은 CollectionModel로 묶어야 함.

```
public Iterable <Taco> recentTacos () {
```

```
    PageRequest page = PageRequest.of (
```

```
        0, 12, Sort.by ("createdAt").descending ();
```

```
    return tacoRepo.findAll (page).getContent ();
```

```
}
```

```
}
```

↳ spring boot 2.x 버전

spring boot 3.x는 Jakarta EE 기반으로 변경되면서

Spring HATEOAS도 함께 2.x 이상 버전으로 업그레이드.

Iterable <>

Resource <Resource <Taco>>

↳ CollectionModel <EntityModel <Taco>>

Resource, Resources
EntityLinks

→ EntityModel
CollectionModel
WebMvcLinkBuilder

entityLinks.linkToSingleResource (...)

↳ linkTo (methodName (...)) 방식

다시 정리. ... recentTacos()

PageRequest page = PageRequest.of(0, 12, Sort.by("createdAt").descending());
페이지 번호 데이터 개수: 12개.
내림차순 정렬.

List<EntityModel<Taco>> tacoResources = tacos.stream() ... 리스트 변환케터.
→ .stream().map()

.map(taco) → EntityModel.of(taco) (객체 + 링크) 포장.
→ 메소드 URL 링크 생성.
linkTo(methodOn(DesignTacoController.class).recentTacos())
.withSelfRel()).collect(Collectors.toList());

return CollectionModel.of(tacoResources,
linkTo(methodOn(DesignTacoController.class).recentTacos()).withSelfRel());
전체
포장 리스트 응답화.

메소드 내부 구성요소, 사용법.

- PageRequest.of(int page, int size, Sort sort)

: 페이지 요청 만들기 위한 객체.

: 몇 번째 페이지인지, 몇 개씩 가져올 것인지, 어떻게 정렬

- Sort.by("판명").descending()

: 정렬 기준 지정

: 어떤 필드 기준으로 정렬할지? 오름 / 내림

- tacoRepo.findAll(Pageable page).getContent()

: 페이지 요청이 맞게 DB에서 항목 조회 ↳ 실제 리스트 데이터만 쿼리.

: DB에서 12개만 가져옴.

- .stream()

: 리스트 순회하며 가공 가능하게 만든 Java 기능

: 줄 새겨서 개씩 처리

- .map (람다식)

: 리스트 요소 각각 변환

: 하나하나 꺼내서 변환

=> .map (taco -> EntityModel.of(taco, 링크))

: Taco -> EntityModel<Taco>로 변환

- EntityModel.of (Object, Link...)

: 하나씩 객체를 감싸고 HATEOS 링크 붙이는 포장지.

=> EntityModel.of (taco, linkTo(...), withSelfRel())

- linkTo (methodOn(...))

: 현재 컨트롤러의 특정 메소드의 URL 만들어줌.

: 메소드가 실행되는 주소를 링크로 만들어줌.

=> linkTo (methodOn (DesignTacoController.class), recentTacos())

- .withSelfRel()

: 링크에 자기 자신 가리킴.

: JSON 응답에서 _links.self.href로 표시.

=> linkTo(...).withSelfRel()

- collect (Collectors.toList())

: stream으로 처리한 결과 다시 List로 묶음

- CollectionModel.of (List<EntityModel<T>>, Link...)

: 여러 개의 EntityModel을 하나로 묶어서 응답.

여기서 개인적인 의견

2) SQL 문시 Limit 와 Order by 로 특정개수만큼 데이터 가져오고
정렬 시켜서 가져오면 안되나!

: recentTacos() 메소드는 단순 조회 이상이 일을 하기위해서이다.

① HATEOAS 구조

: REST API 가 응답할 때 링크까지 포함된 구조 보여주기 위해.

② Spring Data JPA + 페이징 연동

: PageRequest 는 JPA 내부에서 페이징 자동처리.

③ 확장성 / 재사용성

: 아후, 페이지 이동, 다른 페이지, 상세 조회 링크 붙이기 등 좋음.

④ 포장하고 싶은 것

: 각 항목에 링크가 필요한 것.

ex)

- 단순 SQL 방식 (LIMIT 12)

: Fast, EZ

: RESTful 링크 x, 페이지 이동 x, 재사용성 ↓

- PageRequest + HATEOAS

: 다음 페이지에 대한 정보 포함

: 항목마다 링크 가능

: JSON API 연결 가능

: 코드 길고 어려움.

④ Rest Controller

- : 컨트롤러: 모든 HTTP 요청 처리 메소드에서 HTTP 응답 몸체에 직접 쓰는 값을 반환한다는 것을 Spring 이 알려줌.

Controller 이 View를 통해 HTML로 반환되는 것과 달리,
직접 HTTP 응답으로 브라우저에 전달되어 나타남.

④ Rest Controller = ④ Controller + ④ Response Body

④ Controller도 구현 가능하지만, 모든 요청처리 메소드에 ④ Response Body
저장해야함.

- ④ Request Mapping (path = "/design", produces = "application/json")

⋮
/design 으로 시작하는
요청 경로를 처리

⋮
이 컨트롤러는 JSON 응답 만들어냄.

→ 컨트롤러가 어떤 타입의 데이터를 응답할건지 선언
클라이언트가 요청할 때 Accept 헤더보고
거기에 맞는 타입만 응답하도록 제한.

produces = 값은 String 배열로 저장.

∴ 여러 콘텐츠 타입 지정가능

↳ ..produces = { "application/json", "text/xml" }

- 자주쓰는 콘텐츠 타입 (MIME type)

: application/json

: JSON 응답 (REST API에서 가장 일반적)

: application/xml

: XML 형식

: text/plain

: 그냥 텍스트 응답

: application/octet-stream

: 바이너리 파일 (다크로드 등)

: text/html

: HTML (브라우저용 응답)

- @CrossOrigin

: CORS (Cross-Origin Resource Sharing)

= 다른 도메인 (Origin) 에서 온 요청 허용할 거 말지 결정하는
브라우저 보안 정책.

브라우저가 다른 출처 (서버주소, 포트, 프로토콜, ... 이 다른 서버)
에서 온 요청은 기본적으로 막아버림.

허용 => @CrossOrigin (origins = " * ")
: 모든 출처에서 오는 요청 허용.

특정 도메인 허용 => origins = " http://localhost : 3000 ")

- 타코 ID로 특정 타코만 가져오는 엔드 포인트

=> 메소드 경로에 플레이스 홀더 변수 지정.

@GetMapping ("/{id}")

PathVariable에 의해 {id} 플레이스 홀더가 대응되는 id 매개변수에
해당 요청이 실제 라
들어간다.

public Taco tacoById (@PathVariable ("id") Long id) {

Optional <Taco> optTaco = tacoRepo. findById (id);

if (optTaco.isPresent())

return optTaco.get();

return null

}

Optional : 어떤 값이 있을수도 있고
있을수도 없는 상황에서
사용하는 컨테이너 클래스.
Null일수도 있는 상황에
NullPointerException 방지하고
안전하게 처리.

값이 존재하는 거
확인.

- Optional Method.

isPresent() : 값이 존재하는 거 확인.

get() : 값을 꺼냄. 없으면 예외

orElse(default) : 값이 없을경우 기본 값 반환. opt.orElse("기본")

orElseGet(supplier) : 값 없을 때 함수로 기본 값 생성. opt.orElseGet() -> "기본"

orElseThrow() : 값 없으면 예외발생. opt.orElseThrow()

ifPresent(Consumer) : 값 있으면 처리. opt.ifPresent(System.out::println)

map(Function) : 값 있으면 가공해서 Optional 반환. opt.map(String::toUpperCase)

1. 2 서버에서 데이터 전송하기

API에서 쿼리 생성된 데이터 반환 가능.

API 테스트 => 테스트 데이터 사용하는 것이 좋다.

- Angular Code

onSubmit() {
 HttpClient → post() 메소드 호출.

this.httpClient.post(→ API로 데이터 전송하기.

'http://localhost:8080/design'

[this.model], { → model 변수에 저장된 데이터를 API 엔드 포인트로 전송.

headers: new HttpHeaders().set('Content-type', 'application-json'),

}).subscribe(taco => this.cart.addToCart(taco));

this.router.navigate(['/cart']);

}

post 요청 처리

@PostMapping(consumes = "application/json")

application/json 타입으로 보내는 요청 처리.
요청 본문에 JSON 데이터 포함 되어야함.

@ResponseStatus(HttpStatus.CREATED) → 요청 성공 + 요청 결과 리스 생성시 클라이언트에 전달

public Taco postTaco(@RequestBody Taco taco);

return tacoRepo.save(taco);

}

클라이언트가 보내는 JSON 데이터를
Java 객체 자동 변환해서 받음.

만약 @RequestBody가 없다면?

: Spring은 HTTP Body 읽지 않고

URL 쿼리 파라미터 / form-data 읽으려고함.

이게 없다면 null이되어버려서
오류난다.

하지만 null 반환 -> 좋은 방법 x

if) null 반환

client 가 정상 처리 나타내는 HTTP 200 (Ok) 상태코드 받음.
이때는 HTTP 404 (Not Found) 상태코드를 응답으로 반환.

@GetMapping("/{id}")

```
public ResponseEntity<Taco> tacoById(@PathVariable("id") Long id) {
```

```
    Optional<Taco> optTaco = tacoRepo.findById(id);
```

```
    if (optTaco.isPresent()) {
```

```
        return new ResponseEntity<>(optTaco.get(), HttpStatus.OK);
```

```
    }
```

```
    return new ResponseEntity<>(null, HttpStatus.NOT_FOUND);
```

```
}
```

: 타코 있으면 HTTP 200 (Ok)

: 없으면 HTTP 404 (Not Found)

ResponseEntity

: HTTP 응답 전체를 직접 구성할 수 있게 해주는 클래스

Body, Status, Headers 가지 제어 가능하다.

ResponseEntity <T> → 응답에 포함될 데이터 타입.

본문 데이터 (Body)

상태코드 (Http Status)

응답 헤더 (HttpHeaders)

```
return new ResponseEntity<>(data, HttpStatus.OK);
```

JSON으로 응답할 객체

상태코드

```
return ResponseEntity.ok(data)
```

```
new ResponseEntity.notFound().build();
```

```
return ResponseEntity
```

```
    .status(HttpStatus.CREATED)
```

```
    .header("Location", "/new-resource")
```

```
    .body(data);
```


3 서버 데이터 변경하기

데이터 변경 위한 HTTP Method

→ PUT, PATCH.

1) PUT

GET과 반대의 의미. GET: Server → Client로 데이터 전송.
PUT: Client → Server로 데이터 전송.

PUT는 데이터 전체를 교체한다.

2) PATCH

: 일부분 변경.

PUT 메소드 같은 경우, 전체 데이터를 PUT 요청으로 제출해야 한다.
만약, 어느 속성이 생략된다면 속성이 값은 null로 변경된다.
∴ 전체 데이터 보내야 함. 그렇지 않으면 삭제된다.
PUT 요청.

```
@PutMapping("/{orderId}") → 기본 값으로 여러 타입 받는 거 가능.  
public Order putOrder(@RequestBody Order order) {  
    return repo.save(order);  
}
```

부분 변경 요청은 보통 JSON/요만 한다.

```
@PatchMapping(path = "/{orderId}", consumes = "application/json")  
public Order patchOrder(@PathVariable("orderId") Long orderId,  
    @RequestBody Order patch) {  
    Order order = repo.findById(orderId).get();  
    if (patch.getDeliveryName() != null)  
        order.setDeliveryName(patch.getDeliveryName());  
    ...  
    return repo.save(order);  
}
```

why?

일부분 수정 시
어떤 데이터 고칠 지
정확히 지정해야 함.
∴ JSON/만 사용.

@PutMapping의 consumes 지정 안해도 보통 JSON 기대 값으로 동작.

프론트-백 사전 약속: API 문서로 PUT JSON/만 보내야 함 라고 정해둠.

6.1.4 서버에서 데이터 삭제

: HTTP DELETE 요청으로 삭제.

→ ① Delete Mapping.

① DeleteMapping ("/{orderId}")

② ResponseStatus (code = HttpStatus.NO_CONTENT)

```
public void deleteOrder (@PathVariable("orderId") Long id) {
```

```
    try {
```

```
        repo.deleteById(orderId);
```

```
    } catch (EmptyResultDataAccessException e) {} }
```

3

↳ Spring JDBC/JPA에서 결과가 없을 때 인내는 런타임 에러.

- DB에서 특정 데이터 삭제하려고 했는데 결과가 X.
- 삭제하려는 대상이 있는데 delete 요청함.